Marlon — Technical Onboarding Guide

> **Generated:** March 9, 2026
> **Repository:** Alae213/marlon
> **Live URL:** https://marlon-inky.vercel.app

Table of Contents

1. README / Instruction Files Summary
2. Detailed Technology Stack
3. System Overview and Purpose
4. Project Structure and Reading Recommendations
5. Key Components
6. Execution and Data Flows
7. Dependencies and Integrations
8. Diagrams
9. Testing
10. Error Handling and Logging
11. Security Considerations
12. Other Relevant Observations

---

1. README / Instruction Files Summary

The project includes several important documentation files beyond the standard README:

Primary Documentation

File	Purpose
README.md	Standard Next.js boilerplate (not project-specific)
PRD.md	Full Product Requirements Document
AGENTS.md	AI coding agent instructions and conventions
Test.md	Manual test scenarios and expected behaviors
tasks.md	Current task board and development priorities
memory.md	Project context memo for AI agent continuity

Table 1: Key documentation files

Local Setup Instructions

1. Install dependencies:
   npm install
2. Set up environment variables (create .env.local):
   NEXT_PUBLIC_CONVEX_URL=...
   NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=...
   CLERK_SECRET_KEY=...
   CLERK_WEBHOOK_SECRET=...
3. Run Convex backend (in separate terminal):
   npm run convex:dev
4. Run Next.js frontend:
   npm run dev

⚠ **Important Note:** The `dev` script includes OpenCode AI assistant. To run only the Next.js server, use `next dev` directly.

---

2. Detailed Technology Stack

Core Technologies

Technology	Version	Role
TypeScript	^{}5	Primary language
Next.js	16.1.6	Full-stack React framework (App Router)
React	19.2.3	UI library
Node.js	≥ 18	Runtime (inferred)

Table 2: Core runtime and frameworks

Frontend Stack

- **Styling:** Tailwind CSS v4, shadcn/ui components, Radix UI primitives
- **Animation:** Framer Motion v12.34.5
- **Icons:** Lucide React v0.575.0
- **Fonts:** Geist v1.7.0 (Vercel's default font)
- **Utilities:** class-variance-authority, tailwind-merge, clsx

Backend Architecture

Service	Version	Role
Convex	v1.32.0	Serverless backend + real-time database
Clerk	v6.39.0	Authentication & user management
Chargily Pay	v2.1.0	Payment gateway (Algerian market)
svix	v1.86.0	Webhook signature verification

Table 3: Backend services

Database

- **Type:** NoSQL / Document-based
- **Engine:** Convex (managed, built-in)
- **Real-time:** Yes — WebSocket-based reactive queries
- **Schema:** Defined in convex/schema.ts
- **Multi-tenancy:** Isolated by storeId

Architecture Style

- **Serverless + Reactive:** Convex eliminates traditional REST API layer
- **Multi-tenant SaaS:** Each store isolated by storeId/slug
- **App Router:** Next.js 13+ file-based routing with React Server Components
- **Real-time First:** Components subscribe directly to database queries via WebSockets

Development Tools

- **Package Manager:** npm
- **Linting:** ESLint v9
- **Build:** next build, convex deploy
- **AI Assistants:** OpenCode, Cursor, Gemini, Windsurf configurations present

Deployment

- **Frontend:** Vercel (standard for Next.js)
- **Backend/DB:** Convex Cloud (managed)
- **Configuration:** convex.json, next.config.ts

---

3. System Overview and Purpose

What is Marlon?

Marlon is a **multi-tenant SaaS e-commerce store builder** specifically designed for the **Algerian market**. It functions as a lightweight, localized alternative to platforms like Shopify, tailored to the unique needs of Algerian entrepreneurs.

Target Users

Persona	Profile	Core Need
Solo Merchant	Non-technical seller of handmade goods	Simple store + COD order management
Influencer	Sells merchandise to followers	Fast setup, no transaction fees
Agency Owner	Manages 5-10 client stores	Multi-store dashboard, low per-store cost

Table 4: Target user personas

Problems Solved

- **Platform Complexity:** Existing solutions too complex for non-technical users
- **Payment Friction:** Most platforms require USD, Algerian market needs DZD
- **Missing COD Workflows:** Cash-on-delivery is standard in Algeria
- **Tool Fragmentation:** Consolidates store management, orders, and delivery pricing

Core Features

1. **Store Creation:** Merchants sign up via Clerk, create branded store with unique slug (marlon.app/my-store)
2. **Product Management:** Add/edit products with images, variants, categories, price modifiers
3. **Public Storefront:** Customer-facing catalog at /[slug] with cart, search, category filter
4. **Order Management:** Complete COD workflow with status tracking, call logs, admin notes, audit trail
5. **Delivery Pricing:** Per-wilaya home/office delivery pricing for 58 Algerian provinces
6. **Site Content Editor:** Visual CMS for hero, banners, featured products
7. **Billing System:** Free trial (50 orders or 30 days), then subscription tier
8. **Real-time Notifications:** Merchants see new orders arrive instantly via Convex subscriptions
9. **Payment Integration:** Chargily Pay for DZD transactions

Business Model

Pay-when-you-succeed pricing:

- Free tier: First 50 orders per store per 30-day rolling window
- Paid tier: 2000 DZD/month per store (manual renewal, no auto-billing)
- Payment via Chargily Pay (CIB card, Edahabia, bank transfer)

---

4. Project Structure and Reading Recommendations

Entry Points

File	Purpose
app/layout.tsx	Root layout with all providers (Clerk, Convex, Theme, Toast)
app/page.tsx	Dashboard / landing page (16KB monolithic file)
proxy.ts	Next.js middleware (handles Clerk auth protection)
convex/schema.ts	Database schema — single source of truth for data models

Table 5: Primary entry points

⚠ **Critical Note:** The middleware file is named `proxy.ts` (should be `middleware.ts`). This is an important naming discrepancy.

Directory Structure

```
marlon/
├── app/ # Next.js App Router pages
│ ├── [slug]/ # Public storefront (dynamic store)
│ │ ├── page.tsx # Store homepage (product listing)
│ │ └── product/ # Individual product page
│ ├── editor/
│ │ └── [storeSlug]/
│ │ └── page.tsx # Merchant dashboard (75KB monolithic)
│ ├── api/
│ │ ├── webhooks/ # Clerk webhook handler
│ │ ├── chargily/ # Payment API routes
│ │ ├── delivery/ # Delivery pricing API
│ │ └── orders/ # Order management API
│ ├── orders/ # Order tracking page
│ ├── marketing/ # Marketing/landing page
│ ├── layout.tsx # Root layout
│ ├── globals.css # Global styles
```

```
│  ├── global-error.tsx # Global error boundary
│  └── not-found.tsx # 404 page
│
├── convex/ # Serverless backend
│  ├── schema.ts # Database schema
│  ├── orders.ts # Order queries & mutations
│  ├── products.ts # Product management
│  ├── stores.ts # Store CRUD + billing logic
│  ├── users.ts # User management
│  ├── siteContent.ts # CMS content
│  ├── auth.ts # Convex auth setup
│  └── _generated/ # Auto-generated (don't edit)
│
├── components/ # Shared React components
│  ├── ui/ # shadcn/ui primitives
│  ├── animate-ui/ # Animated variants
│  ├── cart-sidebar.tsx # Shopping cart drawer
│  ├── billing-section.tsx # Subscription status UI
│  ├── delivery-settings.tsx # Delivery config
│  ├── image-cropper.tsx # Image upload tool
│  ├── notification-bell.tsx # Real-time order alerts
│  ├── payment-modal.tsx # Chargily payment flow
│  └── wilaya-select.tsx # Algerian province picker
│
├── contexts/ # React Context providers
│  ├── billing-context.tsx # Subscription state machine
│  ├── cart-context.tsx # Shopping cart state
│  ├── realtime-context.tsx # Real-time notifications
│  └── toast-context.tsx # Toast queue
│
├── hooks/ # Custom React hooks
├── lib/ # Utilities
├── public/ # Static assets
├── .agents/ # AI agent configs
│
├── proxy.ts # Next.js middleware
├── convex.json # Convex config
├── components.json # shadcn/ui config
```

```
├── package.json # Dependencies
└── tsconfig.json # TypeScript config
```

Recommended Reading Order

1. convex/schema.ts — Understand all data entities first
2. PRD.md — Product spec and billing model
3. app/layout.tsx — Provider hierarchy and app shell
4. proxy.ts — Auth routing rules
5. convex/stores.ts — Core business logic
6. convex/orders.ts — Order lifecycle
7. contexts/billing-context.tsx — Subscription state machine
8. contexts/realtime-context.tsx — Real-time architecture
9. app/[slug]/page.tsx — Customer storefront
10. app/editor/[storeSlug]/page.tsx — Merchant dashboard (large file)

---

5. Key Components

5.1 Database Schema (convex/schema.ts)

The complete data model defining all 6 tables:

```
export default defineSchema({
users: defineTable({
clerkId: v.string(),
email: v.string(),
firstName: v.optional(v.string()),
lastName: v.optional(v.string()),
phone: v.optional(v.string()),
theme: v.optional(v.string()),
createdAt: v.number(),
updatedAt: v.number(),
}).index("clerkId", ["clerkId"]),

stores: defineTable({
ownerId: v.string(),
name: v.string(),
slug: v.string(),
subscription: v.optional(v.string()), // "trial" | "active" | "locked"
```

```
orderCount: v.optional(v.number()),
firstOrderAt: v.optional(v.number()),
paidUntil: v.optional(v.number()),
lockedAt: v.optional(v.number()),
// ... other fields
})
.index("ownerId", ["ownerId"])
.index("slug", ["slug"]),

products: defineTable({
storeId: v.string(),
name: v.string(),
basePrice: v.number(),
variants: v.optional(v.array(v.object({
name: v.string(),
options: v.array(v.object({
name: v.string(),
priceModifier: v.optional(v.number()),
})),
})),
isArchived: v.optional(v.boolean()),
sortOrder: v.optional(v.number()),
// ... other fields
})
.index("storeId", ["storeId"]),

orders: defineTable({
storeId: v.string(),
orderNumber: v.string(),
customerName: v.string(),
customerPhone: v.string(),
customerWilaya: v.string(),
products: v.array(v.object({
productId: v.string(),
name: v.string(),
price: v.number(),
quantity: v.number(),
variant: v.optional(v.string()),
})),
status: v.string(),
```

```
timeline: v.optional(v.array(v.object({
status: v.string(),
timestamp: v.number(),
note: v.optional(v.string()),
}))),
callLog: v.optional(v.array(v.object({
id: v.string(),
timestamp: v.number(),
outcome: v.string(),
}))),
adminNotes: v.optional(v.array(v.object({
id: v.string(),
text: v.string(),
timestamp: v.number(),
merchantId: v.string(),
}))),
// ... other fields
})
.index("storeId", ["storeId"])
.index("status", ["storeId", "status"]),

siteContent: defineTable({
storeId: v.string(),
section: v.string(),
content: v.any(),
updatedAt: v.number(),
})
.index("storeId", ["storeId"])
.index("section", ["storeId", "section"]),

deliveryPricing: defineTable({
storeId: v.string(),
wilaya: v.string(),
homeDelivery: v.optional(v.number()),
officeDelivery: v.optional(v.number()),
updatedAt: v.number(),
})
.index("storeId", ["storeId"])
.index("wilaya", ["storeId", "wilaya"]),
});
```

5.2 Store & Billing Logic (convex/stores.ts)

The most business-critical backend file containing:

- CRUD operations for stores
- Delivery pricing management
- Billing state machine implementation

Key Function — Billing State Machine:

```
export const handleNewOrderSubscription = mutation({
args: { storeId: v.id("stores") },
handler: async (ctx, args) => {
// Handles transitions:
// - trial → locked (50 orders reached)
// - active → trial (subscription expired)
// - trial window reset (30 days passed)
// - active order count increment
},
});
```

Subscription States: `"trial"` → `"active"` (paid) or `"locked"` (limit exceeded)

5.3 Order Lifecycle (convex/orders.ts)

Manages complete order workflow:

```
// Creates order + increments store orderCount
export const createOrder = mutation({
args: { storeId, customerData, products, ... },
// Implementation
});

// Status transitions with immutable timeline
export const updateOrderStatus = mutation({
args: { orderId, status, note },
handler: async (ctx, args) => {
const timeline = [
...(order.timeline ?? []),
{ status, timestamp: Date.now(), note }
];
```

```
await ctx.db.patch(args.orderId, {
status,
timeline,
updatedAt: Date.now()
});
},
});
```

Order Status Flow:

$$\text{new} \rightarrow \text{confirmed} \rightarrow \text{packaged} \rightarrow \text{shipped} \rightarrow \text{succeeded}$$

With alternative terminal states: canceled, blocked, hold

5.4 Billing Context (contexts/billing-context.tsx)

Client-side reactive mirror of billing state machine:

```
const isLocked =
storeStatus === "locked" | |
(storeStatus === "trial" && daysRemaining <= 0) | |
(storeStatus === "trial" && currentOrderCount >=
FREE_ORDER_LIMIT); // 50

// Exposed methods
const { openPaymentModal, closePaymentModal } = useBilling();
```

Used by `components/locked-overlay.tsx` to block merchant dashboard when limits exceeded.

5.5 Real-time Context (contexts/realtime-context.tsx)

Delivers live order notifications to merchants:

```
const realtimeOrders = useQuery(
api.orders.subscribeToStoreOrders,
storeId ? { storeId } : "skip"
);

// New orders since last connection
const newOrdersCount = realtimeOrders?.filter(order =>
order.status === "new" &&
```

```
order.createdAt > (lastUpdate || 0)
).length || 0;
```

Powers `components/notification-bell.tsx` with red badge indicator.

5.6 Shopping Cart (contexts/cart-context.tsx)

In-memory cart state for customer storefront:

```
interface CartItem {
productId: string;
name: string;
price: number;
quantity: number;
variant?: string;
image?: string;
}

const {
items,
addItem,
removeItem,
updateQuantity,
clearCart,
isOpen,
openCart,
closeCart
} = useCart();
```

⚠ **Limitation:** Cart is not persisted — page refresh clears it.

5.7 Provider Hierarchy (app/layout.tsx)

Complete provider nesting:

```
// Authentication
// Backend/Database
// Light/dark mode
// Notifications
{children}
```

5.8 Middleware (proxy.ts)

Protects authenticated routes:

```
const isProtectedRoute = createRouteMatcher([
"/dashboard(.)",
"/store(.)"
]);

export default clerkMiddleware(async (auth, req) => {
if (isProtectedRoute(req)) await auth.protect();
});
```

5.9 Wilaya Selector (components/wilaya-select.tsx)

Searchable dropdown with all 58 Algerian provinces. Critical for:

- Customer checkout (delivery address)
- Merchant delivery pricing configuration
- Order management filtering

---

6. Execution and Data Flows

6.1 Customer Order Flow

```
Customer visits /{slug}
↓
app/[slug]/page.tsx
↓ Convex Query
stores.getStoreBySlug(slug)
products.getProducts(storeId)
↓
Products displayed with CartProvider
↓
Customer adds item to cart
↓
CartContext.addItem() → in-memory state
CartSidebar opens
↓
Customer proceeds to checkout
↓
```

DeliveryAction component
↓ Convex Query
stores.getDeliveryCost(storeId, wilaya)
↓
Total = subtotal + deliveryCost
↓
Customer submits order
↓ Convex Mutation
orders.createOrder(storeId, customerData, products, total)
↓
Insert into orders table
↓
patch stores.orderCount++
↓ Convex Mutation
stores.handleNewOrderSubscription(storeId)
↓
Billing state machine evaluates trial/lock conditions
↓
CartContext.clearCart()
↓
Order confirmation page displayed

6.2 Merchant Dashboard Flow

Merchant logs in (Clerk)
↓
proxy.ts middleware validates session
↓
app/page.tsx → redirect to editor/[storeSlug]
↓
app/editor/[storeSlug]/page.tsx
↓
BillingProvider wraps dashboard
RealtimeProvider subscribes to store orders
↓ Convex Subscription (WebSocket)
orders.subscribeToStoreOrders(storeId)
↓
NotificationBell shows new order count in real-time
↓
Merchant views/manages orders

↓ Convex Mutations
orders.updateOrderStatus() → timeline entry appended
orders.addCallLog() → callAttempts++
orders.addAdminNote() → adminNotes[] appended
↓
Merchant edits products
↓ Convex Mutations
products.createProduct()
products.updateProduct()
products.archiveProduct()
products.reorderProducts() → sortOrder batch update
↓
Merchant updates site content
↓ Convex Mutations
siteContent.upsert(storeId, section, content)

6.3 Auth & User Sync Flow

User signs up on Clerk
↓
Clerk fires webhook
↓
app/api/webhooks/
↓
svix verifies CLERK_WEBHOOK_SECRET
↓ Convex Mutation
users.syncUser(clerkId, email, firstName, ...)
↓
Upsert user record in users table
↓
User logs in
↓
Clerk session token attached to all requests
↓
ClerkProvider makes auth state available globally

6.4 Billing State Machine Flow

```
Trial Store: New Order Received
↓
handleNewOrderSubscription()
↓
if (paidUntil < now && subscription === "active")
↓
revert to trial, reset orderCount = 1
↓
if (now - firstOrderAt > 30 days)
↓
reset window: firstOrderAt = now, orderCount = 1
↓
if (orderCount > 50)
↓
subscription = "locked", lockedAt = now
↓
else
↓
orderCount++
↓
Frontend (BillingContext):
↓
isLocked = (status === "locked")
OR (trial && daysRemaining <= 0)
OR (trial && orderCount >= 50)
↓
LockedOverlay rendered over dashboard
↓
openPaymentModal() → PaymentModal → Chargily checkout
```

6.5 Database Schema Overview

Table	Key Fields	Indexes
users	clerkId, email, firstName, lastName	clerkId
stores	ownerId, slug, subscription, orderCount, paidUntil	ownerId, slug
products	storeId, name, basePrice, variants, sortOrder	storeId, category
orders	storeId, orderNumber, status, timeline, callLog	storeId, status, orderNumber
siteContent	storeId, section, content (any)	storeId, section
deliveryPricing	storeId, wilaya, homeDelivery, officeDelivery	storeId, wilaya

Table 6: Database tables and indexes

Relationships:

- users → stores (1:many via ownerId)
- stores → products (1:many via storeId)
- stores → orders (1:many via storeId)
- stores → siteContent (1:many via storeId + section)
- stores → deliveryPricing (1:58 via storeId + wilaya)

---

7. Dependencies and Integrations

7.1 Core Dependencies

Package	Version	Purpose
next	16.1.6	Full-stack React framework
react	19.2.3	UI library
convex	^{}1.32.0	Backend + database
@clerk/nextjs	^{}6.39.0	Authentication
tailwindcss	^{}4	CSS framework
framer-motion	^{}12.34.5	Animation library
lucide-react	^{}0.575.0	Icon library

Table 7: Major dependencies

7.2 External Integrations

Clerk Authentication

- **Method:** Google OAuth only (no email/password)
- **Flow:** Landing → "Sign in with Google" → redirect to dashboard
- **Webhook:** Syncs user data to Convex on creation
- **JWT:** Session tokens with auto-refresh
- **Configuration:** Environment variables in .env.local

```
// Webhook endpoint
POST /api/webhooks/clerk

// Verification via svix
const wh = new Webhook(process.env.CLERK_WEBHOOK_SECRET);
wh.verify(payload, headers);

// Sync to Convex
await ctx.runMutation(api.users.syncUser, userData);
```

Chargily Pay Integration

- **Currency:** DZD (Algerian Dinar) only
- **Payment Methods:** CIB card, Edahabia, bank transfer
- **Amount:** 2000 DZD per store per month
- **Flow:** Merchant clicks "Pay Now" → Convex creates invoice → redirect to checkout → webhook confirms payment

```
// Payment flow
```

1. Merchant: openPaymentModal()
2. Convex: createChargilyInvoice(storeId, 2000)
3. Redirect: checkoutUrl (Chargily hosted)
4. Webhook: POST /api/webhooks/chargily
5. Convex: updateStore({ subscription: "active", paidUntil: now+30d })

Convex Real-time Subscriptions

- **Protocol:** WebSocket
- **Reactivity:** Component re-renders on database changes
- **Use Cases:** New order notifications, live order status updates, product catalog changes

```
// Frontend hook
const orders = useQuery(api.orders.subscribeToStoreOrders, { storeId
});

// Backend query (auto-subscribes client)
export const subscribeToStoreOrders = query({
args: { storeId: v.id("stores") },
handler: async (ctx, args) => {
return await ctx.db
.query("orders")
.withIndex("storeId", (q) => q.eq("storeId", args.storeId))
.collect();
},
});
```

7.3 API Documentation

The project uses **Convex's auto-generated API documentation**:

- Access via: Convex Dashboard → Functions tab
- All queries, mutations, and actions auto-documented
- TypeScript types auto-generated in convex/_generated/
- Import from: `import { api } from "@/convex/_generated/api"`

Example API usage:

```
// Frontend
import { useQuery, useMutation } from "convex/react";
```

```
import { api } from "@/convex/_generated/api";

const products = useQuery(api.products.getProducts, { storeId });
const createOrder = useMutation(api.orders.createOrder);

await createOrder({ storeId, customerData, products });
```

---

8. Diagrams

8.1 System Architecture

graph TB
subgraph Client
Browser[Browser/Customer]
Dashboard[Merchant Dashboard]
end

```
subgraph "Next.js App (Vercel)"
    AppRouter[App Router]
    Middleware[proxy.ts Middleware]
    API[API Routes]
end

subgraph "Convex Backend"
    Queries[Queries]
    Mutations[Mutations]
    Database[(Convex DB)]
end

subgraph "External Services"
    Clerk[Clerk Auth]
    Chargily[Chargily Pay]
end

Browser -->|HTTPS| AppRouter
Dashboard -->|WebSocket| Queries
AppRouter --> Middleware
Middleware --> Clerk
```

```
AppRouter --> API
API --> Mutations
Dashboard --> Mutations
Mutations --> Database
Queries --> Database
API --> Chargily
Clerk -->|Webhook| API
Chargily -->|Webhook| API
```

8.2 Data Flow Diagram

sequenceDiagram
participant C as Customer
participant SF as Storefront
participant Cart as CartContext
participant Conv as Convex
participant M as Merchant
participant RT as RealtimeContext

```
C->>SF: Visit /{slug}
SF->>Conv: getStoreBySlug(slug)
SF->>Conv: getProducts(storeId)
Conv-->>SF: Return products
SF-->>C: Display catalog

C->>Cart: Add to cart
Cart-->>C: Show cart sidebar
C->>Cart: Proceed to checkout
Cart->>Conv: getDeliveryCost(wilaya)
Conv-->>Cart: Return cost
C->>Conv: createOrder(orderData)
Conv->>Conv: Increment orderCount
Conv->>Conv: handleNewOrderSubscription()
Conv-->>C: Order confirmation

Conv->>RT: Broadcast new order
RT-->>M: Show notification
```

```
M->>Conv: Query order details
Conv-->>M: Return order data
```

8.3 Billing State Machine

stateDiagram-v2
[*] --> Trial: Store created
Trial --> Locked: orderCount >= 50
Trial --> Trial: orderCount < 50
Locked --> Active: Payment received
Active --> Trial: Subscription expired
Active --> Active: Order placed (within period)

```
note right of Trial
    orderCount: 0-49
    firstOrderAt: timestamp
    30-day rolling window
end note

note right of Locked
    Public storefront: live
    Orders: masked
    Dashboard: restricted
end note

note right of Active
    paidUntil: now+30d
    orderCount: 0
    No order limit
end note
```

8.4 Component Hierarchy

graph TD
Root[app/layout.tsx]
Root --> Clerk[ClerkProvider]
Clerk --> Convex[ConvexClientProvider]
Convex --> Theme[ThemeProvider]

Theme --> Toast[ToastProvider]
Toast --> Page[app/page.tsx]

```
Page --> Editor[app/editor/[storeSlug]/page.tsx]
Page --> Storefront[app/[slug]/page.tsx]

Editor --> Billing[BillingProvider]
Editor --> Realtime[RealtimeProvider]

Storefront --> Cart[CartProvider]

Billing --> LockedOverlay[LockedOverlay]
Billing --> PaymentModal[PaymentModal]

Realtime --> NotificationBell[NotificationBell]

Cart --> CartSidebar[CartSidebar]
```

8.5 Database Relationships

erDiagram
USERS ||--o{ STORES : owns
STORES ||--o{ PRODUCTS : contains
STORES ||--o{ ORDERS : receives
STORES ||--o{ SITE_CONTENT : has
STORES ||--o{ DELIVERY_PRICING : configures

```
USERS {
    string clerkId PK
    string email
    string firstName
    string lastName
}

STORES {
    id _id PK
    string ownerId FK
```

```
    string slug UK
    string subscription
    number orderCount
    number paidUntil
}

PRODUCTS {
    id _id PK
    string storeId FK
    string name
    number basePrice
    array variants
    number sortOrder
}

ORDERS {
    id _id PK
    string storeId FK
    string orderNumber UK
    string status
    array timeline
    array callLog
}

SITE_CONTENT {
    id _id PK
    string storeId FK
    string section
    any content
}

DELIVERY_PRICING {
    id _id PK
    string storeId FK
    string wilaya
    number homeDelivery
```

```
    number officeDelivery
}
```

---

9. Testing

Current Test Coverage

⚠ **Important:** The project currently has **no automated test suite**. Testing is performed manually using the guidelines in `Test.md`.

Manual Testing Documentation

The `Test.md` file contains detailed manual test scenarios including:

- Store creation flow
- Product management (CRUD operations)
- Order placement from customer perspective
- Order status transitions in merchant dashboard
- Call log functionality
- Billing state transitions
- Payment flow with Chargily
- Real-time notification behavior
- Delivery pricing calculations

Testing Commands

Command	Purpose
npm run lint	Run ESLint checks
npm run build	Production build (catches type errors)
npm run dev	Development server for manual testing

Table 8: Available testing commands

Recommended Testing Strategy

For future implementation:

1. **Unit Tests:** Jest + React Testing Library for components
2. **Integration Tests:** Convex backend functions

3. **E2E Tests:** Playwright for critical user flows
4. **CI/CD:** GitHub Actions on pull requests

---

10. Error Handling and Logging

Error Handling Patterns

Global Error Boundary

```
// app/global-error.tsx
export default function GlobalError({ error, reset }: {
error: Error & { digest?: string };
reset: () => void;
}) {
return (
Something went wrong!

<button onClick={() => reset()}>Try again</button>
);
}
```

Convex Error Handling

Convex mutations and queries use standard try-catch patterns:

```
export const createOrder = mutation({
handler: async (ctx, args) => {
try {
// Validation
if (!args.storeId) {
throw new Error("Store ID is required");
}

  // Business logic
  const orderId = await ctx.db.insert("orders", orderData);
  return { success: true, orderId };

} catch (error) {
  console.error("Failed to create order:", error);
```

```
    throw new Error("Failed to create order");
  }
```

},
});

Client-Side Error Handling

```
// Using toast notifications for user-facing errors
const createOrder = useMutation(api.orders.createOrder);

try {
await createOrder(orderData);
toast.success("Order placed successfully!");
} catch (error) {
toast.error("Failed to place order. Please try again.");
console.error(error);
}
```

Logging Strategy

Current Implementation:

- **Development:** Console.log statements throughout codebase
- **Production:** Vercel and Convex platform logs
- **No structured logging:** No Winston, Pino, or similar library

Audit Trail:

The application implements immutable audit logging for orders:

```
// All order changes logged in auditTrail array
{
auditTrail: [
{
id: "uuid",
timestamp: Date.now(),
action: "status_changed",
details: "new -> confirmed",
},
{
```

```
id: "uuid",
timestamp: Date.now(),
action: "call_attempted",
details: "outcome: answered",
},
],
}
```

Monitoring

Available through platforms:

- **Vercel:** Real-time logs, build logs, function logs
- **Convex Dashboard:** Function execution logs, query performance, error rates
- **Clerk Dashboard:** Authentication events, user activity

⚠ **No dedicated monitoring tool** (e.g., Datadog, Sentry) currently integrated.

---

11. Security Considerations

Authentication

- **Provider:** Clerk (Google OAuth only)
- **Session Management:** JWT tokens with auto-refresh
- **Token Storage:** HTTP-only cookies managed by Clerk
- **Middleware Protection:** proxy.ts enforces auth on protected routes

```
// Route protection
const isProtectedRoute = createRouteMatcher([
"/dashboard(.)",
"/store(.)"
]);

export default clerkMiddleware(async (auth, req) => {
if (isProtectedRoute(req)) {
await auth.protect();
}
});
```

Authorization

Multi-tenant isolation enforced at database query level:

```
// All queries filtered by user's stores
const userStores = await ctx.db
.query("stores")
.withIndex("ownerId", (q) => q.eq("ownerId", userId))
.collect();

// Users cannot access other users' data
if (store.ownerId !== userId) {
throw new Error("Unauthorized");
}
```

Input Validation

Client-side validation:

- Phone number format validation (Algerian format)
- Email validation via Clerk
- Form field validation with controlled inputs

Server-side validation:

- Convex schema enforces type safety
- Manual validation in mutation handlers
- No explicit validation library (e.g., Zod, Yup)

Secrets Management

Secret	Storage Method
NEXT_PUBLIC_CONVEX_URL	Environment variable (public)
CLERK_SECRET_KEY	Environment variable (server-only)
CLERK_WEBHOOK_SECRET	Environment variable (server-only)
CHARGILY_API_KEY	Environment variable (server-only)
Delivery API credentials	Encrypted in Convex database

Table 9: Secret storage methods

⚠ **No secrets manager** (e.g., AWS Secrets Manager, Vault) currently used.

Webhook Security

Clerk webhooks:

```
import { Webhook } from "svix";

const wh = new Webhook(process.env.CLERK_WEBHOOK_SECRET!);
wh.verify(payload, headers); // Throws on invalid signature
```

Chargily webhooks:

```
// Signature verification via Chargily SDK
const isValid = chargily.verifyWebhookSignature(
payload,
headers["x-signature"]
);

if (!isValid) {
throw new Error("Invalid webhook signature");
}
```

Security Headers

Configured in `next.config.ts`:

```
{
headers: async () => [
{
source: "/(.*)",
headers: [
{ key: "X-Content-Type-Options", value: "nosniff" },
{ key: "X-Frame-Options", value: "DENY" },
{ key: "X-XSS-Protection", value: "1; mode=block" },
],
},
],
}
```

Data Privacy

Locked store behavior:

- Customer PII (name, phone, address) masked with `***` when store locked
- Data never deleted — only masked in UI
- Full access restored upon payment

Known Security Gaps

- No rate limiting on API routes
- No CSRF protection beyond Next.js defaults
- No Content Security Policy (CSP) headers
- No input sanitization library
- Delivery API credentials encrypted but encryption method not specified in code

---

12. Other Relevant Observations

12.1 Build and Deployment

Build Configuration:

```
// package.json scripts
{
"build": "next build",
"start": "next start",
"convex:deploy": "npx convex deploy"
}
```

No containerization:

- No Dockerfile present
- Not using Docker Compose
- Deployed directly to Vercel (Next.js) and Convex Cloud (backend)

Deployment Strategy:

1. Git push to main branch
2. Vercel auto-deploys frontend (continuous deployment)

3. Convex backend deployed via: `npm run convex:deploy`
4. No staging environment visible in config

12.2 AI Coding Assistant Configuration

The project includes extensive AI assistant configurations:

Directory	Purpose
.agents/	Codex AI instructions
.gemini/	Google Gemini AI config
.opencode/	OpenCode AI settings
.windsurf/	Windsurf AI config

Table 10: AI assistant configurations

AGENTS.md contains detailed coding conventions and architecture decisions for AI agents.

12.3 Performance Considerations

Optimizations implemented:

- **Images:** Next.js Image component with lazy loading
- **Fonts:** Geist font optimized by Vercel
- **Code Splitting:** Automatic via Next.js App Router
- **Real-time:** Convex subscriptions minimize redundant queries

Performance targets (from PRD):

- Page load: $<$ 2s LCP
- API response: $<$ 500ms p95
- Real-time updates: $<$ 100ms
- Uptime: 99.9%

12.4 Known Technical Debt

From code analysis and documentation:

1. **Monolithic Pages:**
   - app/page.tsx (16KB)

- app/editor/[storeSlug]/page.tsx (75KB)
- Should be refactored into smaller components

2. **Cart Persistence:**
   - Shopping cart not persisted
   - Lost on page refresh
   - Should use localStorage or Convex
3. **Middleware Naming:**
   - proxy.ts should be middleware.ts
   - Works but violates Next.js conventions
4. **No Automated Tests:**
   - Manual testing only
   - High risk for regressions
5. **Type Safety Gaps:**
   - Some components use `any` types
   - siteContent table uses `v.any()`
6. **No Error Monitoring:**
   - No Sentry or similar
   - Errors only visible in platform logs

12.5 Browser Compatibility

Supported browsers (from PRD):

- Chrome, Firefox, Safari, Edge (last 2 versions)
- iOS Safari
- Chrome Mobile

No IE11 support — uses modern JavaScript features.

12.6 Localization

Current state:

- UI is English only
- Algerian-specific features: wilaya selector, DZD currency
- Phone number validation for Algerian format
- Arabic/French UI not implemented (marked as out of scope for MVP)

12.7 Code Quality Tools

Tool	Configuration
ESLint	eslint.config.mjs (v9)
TypeScript	tsconfig.json (strict mode enabled)
Prettier	Not configured
Husky	Not configured (no pre-commit hooks)

Table 11: Code quality tooling

12.8 Documentation Quality

Strengths:

- Comprehensive PRD with detailed feature specifications
- AI agent instructions (AGENTS.md) document architecture decisions
- Manual test scenarios documented (Test.md)
- Task board maintained (tasks.md)

Gaps:

- No API documentation beyond auto-generated Convex types
- No architecture decision records (ADRs)
- README.md is boilerplate
- No contribution guidelines

12.9 Scalability Considerations

Current limitations:

- **Convex free tier:** Limited to 1GB storage, 1M function calls/month
- **Vercel free tier:** Bandwidth and build minute limits
- **No caching layer:** All queries hit database
- **No CDN for images:** Images served from Convex storage
- **No database indexing review:** May need optimization at scale

Scalability strengths:

- Serverless architecture scales automatically

- Multi-tenant design with proper isolation
- Real-time subscriptions don't poll database
- Static assets deployed to Vercel edge network

Conclusion

This document provides a comprehensive technical overview of the Marlon e-commerce platform. New developers should:

1. Read PRD.md to understand business requirements
2. Study convex/schema.ts to understand data models
3. Explore billing state machine in convex/stores.ts
4. Review real-time architecture in contexts/
5. Examine customer flow in app/[slug]/page.tsx
6. Study merchant dashboard in app/editor/[storeSlug]/page.tsx

For questions or clarifications, refer to:

- AGENTS.md for coding conventions
- Test.md for manual testing procedures
- tasks.md for current development priorities
- Convex Dashboard for backend function documentation

⚠ Critical reminders:

- The middleware file is named proxy.ts (non-standard)
- Shopping cart is not persisted across page reloads
- No automated tests — manual testing required
- Monitor Convex and Vercel usage to stay within free tier limits